

第四章—语义分析

编译程序的目标是将源程序翻译成等价的目标程序，不仅语法正确，而且含义也要正确。



4.1 语义分析概述

语义是源程序及其组成部分所表述的含义。

语义是关于语法结构的含义及其使用规则的描述。

语法正确的程序，语义不一定正确。

语义分析的示例

```
// 示例C代码。有哪些错误？  
void f (int *p)  
{  
    x += 4;  
    p (23);  
    "hello" + "world";  
}  
  
int main ()  
{  
    f () + 5;  
    break;  
    return;  
}
```

程序设计语言的语义分为静态语义和动态语义两种：静态语义是在编译阶段能够检查的语义，如：标识符未定义、类型不匹配等

4.1 语义分析概述

- 静态语义检查：如：类型、一致性、控制流等的检查
 - 动态语义检查：如：除数为0、无效指针、数组下标越界等。
 - 动态语义主要指逻辑含义，在运行时检查的。
 - 编译阶段主要是静态语义检查。
- 

4.1 语义分析概述

语义分析的任务：对语法正确的源程序进行上下文有关性质的审查，审查源程序有无语义错误，为代码生成阶段收集类型信息。语法分析在编译器中的位置：



- 输入

- 语法分析单位

- 输出

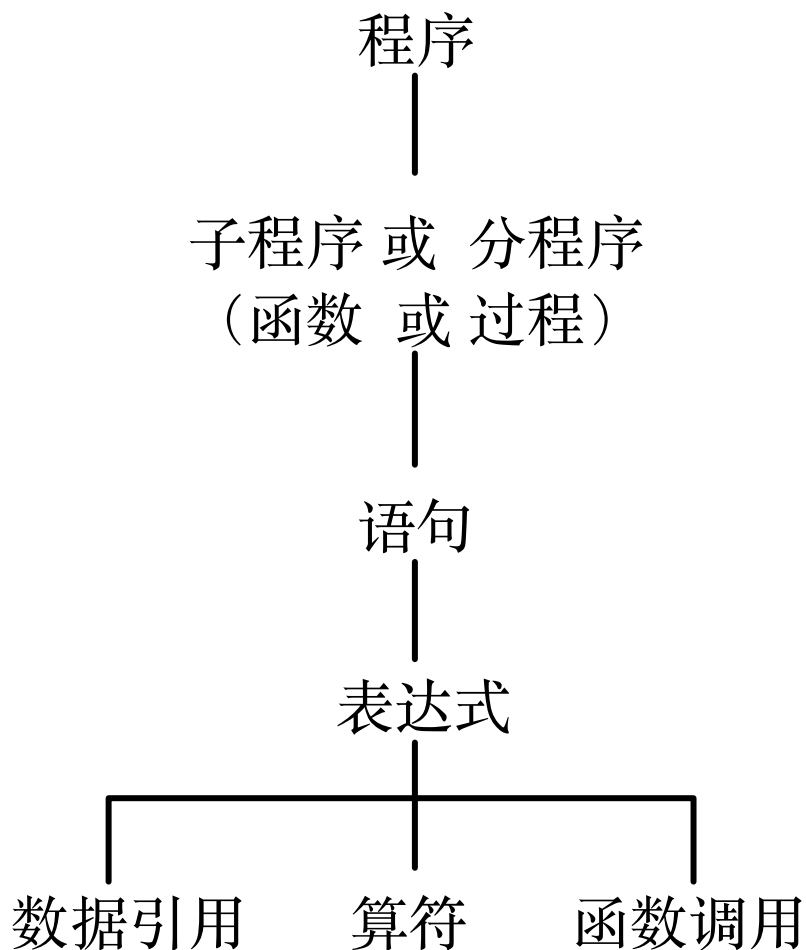
- 用中间代码形式表示的文本
 - 出错处理：定位、继续编译

- 语义分析程序对语法树进行分析，遍历语法树的各个节点，读到声明语句，向符号表中添加信息；读到执行语句，需要检查标识符与符号表中存储的属性是否一致，不一致报告语义错误，一致则生成中间代码。



4.2 语言的语义描述

1 程序结构



4.2 语言的语义描述

2 程序

- 程序是语言中最大的语法单位。设计时应考虑程序的总体结构，各子程序之间的关系，如何构成一个程序等。

$\langle \text{程序} \rangle \rightarrow \text{program } \langle \text{标识符} \rangle; \langle \text{程序体} \rangle$

$\langle \text{程序体} \rangle \rightarrow \langle \text{常量说明} \rangle \langle \text{变量说明} \rangle \langle \text{复合句} \rangle.$

4.2 语言的语义描述

3 语句

程序由各种语句构成。

- 语句
 - 说明性语句
 - 可执行语句
- 说明性语句定义了各种不同数据类型的变量或运算，不需要由编译程序生成目标代码，主要用来告诉编译程序一些实体的属性，供编译程序生成目标代码时使用。
- 可执行语句旨在描述程序的动作，需要由编译程序生成目标代码来实现它的语义。

4.2 语言的语义描述

(1) 说明语句

$\langle \text{说明语句} \rangle \rightarrow \langle \text{常量说明} \rangle | \langle \text{变量说明} \rangle$

$\langle \text{常量说明} \rangle \rightarrow \text{const } \langle \text{标识符} \rangle = \langle \text{常数} \rangle$

$\langle \text{变量说明} \rangle \rightarrow \text{var } \langle \text{变量表} \rangle : \langle \text{类型} \rangle$

$\langle \text{变量表} \rangle \rightarrow \langle \text{变量} \rangle | \langle \text{变量表} \rangle, \langle \text{变量} \rangle$

$\langle \text{变量} \rangle \rightarrow \langle \text{标识符} \rangle$

$\langle \text{类型} \rangle \rightarrow \text{integer} | \text{real} | \text{char} | \text{boolean}$

4.2 语言的语义描述

声明语句的语义

变量声明语句的含义是定义一个存储空间并规定该存储空间的大小，存放数据的类型、取值范围以及该数据进行的操作，变量是数据的符号化形式。

变量声明语句声明程序中使用的变量的名字和类型，变量声明可以给变量赋初值。

常量声明语句的含义是定义一个名字、常量的类型以及常量的值，但常量不能出现在赋值运算的左边。

4.2 语言的语义描述

(2) 执行语句

$\langle \text{执行语句} \rangle \rightarrow \langle \text{读语句} \rangle | \langle \text{写语句} \rangle | \langle \text{赋值语句} \rangle | \langle \text{控制语句} \rangle | \langle \text{复合语句} \rangle$

$\langle \text{读语句} \rangle \rightarrow \text{read} (\langle \text{变量} \rangle)$

$\langle \text{写语句} \rangle \rightarrow \text{write} (\langle \text{变量} \rangle)$

$\langle \text{赋值语句} \rangle \rightarrow \langle \text{变量} \rangle = \langle \text{表达式} \rangle$

$\langle \text{表达式} \rangle \rightarrow \langle \text{算术表达式} \rangle | \langle \text{逻辑表达式} \rangle | \langle \text{关系表达式} \rangle$

$\langle \text{控制语句} \rangle \rightarrow \langle \text{if语句} \rangle | \langle \text{while语句} \rangle | \langle \text{for语句} \rangle | \langle \text{dowhile语句} \rangle$

$\langle \text{复合语句} \rangle \rightarrow \text{begin} \langle \text{语句表} \rangle \text{end}$

.....

4.2 语言的语义描述

4 表达式

表达式是构成程序设计语言中语句的最基本的成分。

- 表达式由运算对象和运算符组成。
- 分为逻辑表达式、关系表达式和算术表达式
- 运算符之间的优先关系和结合性规定了表达式的计算次序。
- 算术表达式的含义是进行两个运算对象的算术运算，关系表达式的运算结果是一个布尔值。

4.2 语言的语义描述

(1) 逻辑表达式

$\langle \text{逻辑表达式} \rangle \rightarrow \langle \text{布尔常量} \rangle$

$\langle \text{逻辑表达式} \rangle \rightarrow \langle \text{布尔变量} \rangle$

$\langle \text{逻辑表达式} \rangle \rightarrow (\langle \text{关系表达式} \rangle)$

$\langle \text{逻辑表达式} \rangle \rightarrow \text{not } \langle \text{逻辑表达式} \rangle$

$\langle \text{逻辑表达式} \rangle \rightarrow \langle \text{逻辑表达式} \rangle \text{ and } \langle \text{逻辑表达式} \rangle$

$\langle \text{逻辑表达式} \rangle \rightarrow \langle \text{逻辑表达式} \rangle \text{ or } \langle \text{逻辑表达式} \rangle$

4.2 语言的语义描述

(2) 关系表达式

$\langle \text{关系表达式} \rangle \rightarrow \langle \text{算术表达式} \rangle \langle \text{关系运算符} \rangle \langle \text{算术表达式} \rangle$

$\langle \text{关系运算符} \rangle \rightarrow <|>|<>|<=|>=|==$

4.2 语言的语义描述

(3) 算术表达式

$\langle \text{算术表达式} \rangle \rightarrow \langle \text{常量} \rangle | \langle \text{变量} \rangle | (\langle \text{算术表达式} \rangle) |$
 $\langle \text{算术表达式} \rangle \langle \text{算术运算符} \rangle \langle \text{算术表达式} \rangle$

$\langle \text{算术表达式} \rangle \rightarrow \langle \text{算术表达式} \rangle + \langle \text{项} \rangle$
 $| \langle \text{算术表达式} \rangle - \langle \text{项} \rangle | \langle \text{项} \rangle$

$\langle \text{项} \rangle \rightarrow \langle \text{项} \rangle * \langle \text{因子} \rangle | \langle \text{项} \rangle / \langle \text{因子} \rangle | \langle \text{因子} \rangle$

$\langle \text{因子} \rangle \rightarrow (\langle \text{算术表达式} \rangle) | \langle \text{常量} \rangle | \langle \text{变量} \rangle$

$\langle \text{变量} \rangle \rightarrow \langle \text{标识符} \rangle$

$\langle \text{常量} \rangle \rightarrow \langle \text{整型常量} \rangle | \langle \text{实型常量} \rangle$

• 4.3 符号表管理技术

1 符号表概述：

符号表是一种数据结构，用于保存源程序中出现的名字及其相关的属性信息。

一个名字具有一系列的属性值。

编译器在分析阶段收集信息放入符号表中，在综合阶段生成目标代码时引用名字及其相关属性，查符号表获得。

符号表管理和语义分析、代码生成没有绝对的优先顺序，可以穿插进行。

符号表管理主要是指管理高级语言中的各种名字，包括变量管理、函数管理、字符串常量管理。

• 4.3 符号表管理技术

2 符号表的组织方式:

程序设计语言中有很多不同类型的名字，如常量、变量、函数、字符串、标号、类型等，以及在面向对象程序中设计的类，不同类型的名字具有不同的属性，可以单独为每类名字建立一个符号表，也可以将各种名字放在一张符号表。

入口	单词名	长度	类型	种属	值	内存地址
1	a	1	未知	未知	未知	未知
2	b2	2	未知	未知	未知	未知
3	test	4	未知	未知	未知	未知
.....						

• 4.3 符号表管理技术

3 符号表的实现方法

sym-table

入口	单词名字及长度			类型	种属	值	内存地址
1			1	整	简单变量	未知	未知
2			2	整	简单变量	未知	未知
3			4	实	简单变量	未知	未知
.....							

a	\0	b	2	\0	t	e	s	t	\0	
---	----	---	---	----	---	---	---	---	----	-------

lex

源程序

```
/*this is a sample
  program writing in
  simple language*/
program example1;
/*used for illustrating
  compiling process*/
var
  a,b,c:integer;
  x:char;
begin
  if (a+c*3 > b) and (b>3)
    then c:=3;
End.
```

符号表

num	token	name	type	kind	val	addr
-----	-------	------	------	------	-----	------

(1,	24,	example1		)		
-----	-----	----------	--	---	--	--

(2,	34,	a		)		
-----	-----	---	--	---	--	--

(3,	34,	b		)		
-----	-----	---	--	---	--	--

(4,	34,	c		)		
-----	-----	---	--	---	--	--

(5,	34,	x		)		
-----	-----	---	--	---	--	--

(6,	35,	3		)		
-----	-----	---	--	---	--	--



• 4.3 符号表管理技术

4 符号表的操作

- 符号表的操作主要包括以下几种：
 - 插入操作
 - 查询操作
 - 访问符号表
 - 修改符号表
 - 删除符号表

4.4 静态语义检查

静态语义检查：类型、控制流、一致性、相关名字等的检查

- 类型检查。主要检查操作数和操作符是否相容。
- 控制流检查。用以保证控制语句有合法的转向点。
- 一致性检查。多数语言要求对象只能被定义一次。
- 相关名字检查。有的语言同一名字必须出现两次或者多次。

4.4 静态语义检查

4.4.1 静态语义检查概述

1 相关名字检查

名字使用前应该先声明

语义分析的示例

// 示例C代码。有哪些错误？

```
void f (int *p)
```

```
{  
    x += 4;  
    p (23);  
    "hello" + "world";  
}
```

```
int main ()
```

```
{  
    f () + 5;  
    break;  
    return;  
}
```

4.4 .1 静态语义检查概述

2 类型检查

变量声明的处理

// 类型合法的程序:

```
int x;  
x+4
```

// 类型合法的程序:

```
bool y;  
false && y
```

// 类型不合法的程序:

```
x + 3
```

// 类型不合法的程序:

```
int x;  
x + false
```

$P \rightarrow D E$
 $D \rightarrow T \mid id$
 $T \rightarrow int \mid bool$
 $E \rightarrow n \mid id \mid true \mid false \mid E + E \mid E \&\& E$

4.4.1 静态语义检查概述

类型检查算法

```
enum type {INT, BOOL};  
Table_t table;
```

```
enum type check_prog (Dec_t d, Exp_t e)  
{  
    table = check_dec (d);  
    return check_exp (e);  
}
```

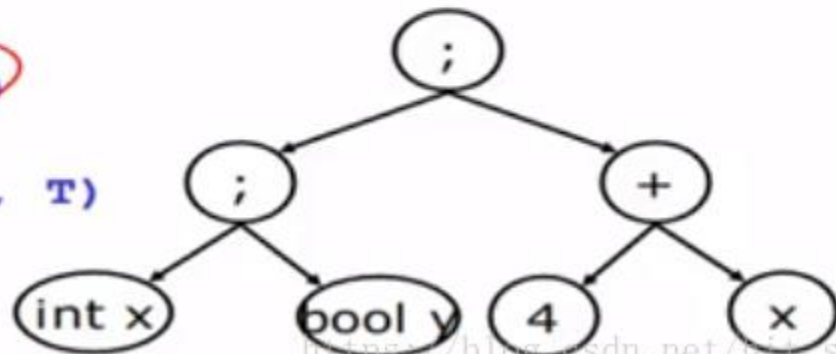
```
Table_t check_dec (Dec_t d)  
{  
    foreach (T id ∈ d)  
        table_enter (table, id, T);  
}
```

check_prog P → DE
check_dec D → T id

T → int
| bool
| id
| true
| false
| E + E
| E && E

check_exp E → n
| id
| true
| false
| E + E
| E && E

key → Value
[id → type]



4.4.1 静态语义检查概述

3 控制流检查

用以保证控制语句有合法的转折点。

在C语言中break语句是否出现在循环中或者

switch...case语句内部

continue语句是否出现在循环内部

4.4.1 静态语义检查概述

4 一致性检查

大多数语言中要求对象只能被定义一次。

在相同作用域中标识符只能声明一次

case语句的标号不能相同

枚举类型的元素不能重复出现

函数名和变量名不能相同

静态语义检查工作通常由专门的语义检查子程序完成。语义检查子程序可以由语法分析程序在进行语法分析过程中直接调用，也可以在语法分析后单独一遍进行静态语义检查

4.4.2 声明与定义语义检查



大多数语言中要求先定义再使用。

变量未声明就使用、变量重定义

函数未声明就定义和调用、函数重定义、函数声明

与定义不匹配、函数和变量重名

函数名和变量名不能相同

数组长度是正整数，数组名和变量名、函数名不重

初始化变量值时需要是已知值，变量初始化时类型

4.4.3 表达式语义检查

表达式是高级程序设计语言数据处理的核心。

函数调用时实参与形参的类型是否匹配、参数个数是否相等

有void返回值的函数不能参加表达式的运算

表达式运算中，操作符是否作用于不相容的操作数
数组下标运算只能作用于数组，指针地址访问只作用于指针

表达式的左值和右值

赋值表达式的类型检查

4.4.4 语句语义检查

高级程序设计语言的各个语句都是独立的单位，即各个语句可以在任何地方使用，很少存在上下文相关的信息。但有三个语句比较特殊

break在循环内出现或者switch分支语句内使用
continue在循环内使用
return在函数中使用，与函数返回值匹配

4.5 小结

语义分析:

静态语义检查

动态语义检查

程序设计语言的语义描述

符号表管理技术

